

Arrow Functions, the Basics – Simple Hinglish Summary

Source: <https://javascript.info/arrow-functions-basics>

Main baat

Function banane ka ek aur **bahut chhota aur simple syntax** hai, jo aksar Function Expression se behtar hota hai. Isse **arrow function** kehte hain kyunki isme `=>` (arrow) hota hai.

javascript

```
let func = (arg1, arg2, ..., argN) => expression;
```

Ye ek function `func` banata hai jo arguments leta hai, right side ki `expression` ko evaluate karta hai aur **uska result return** karta hai.

Ye is code ka chhota version hai:

javascript

```
let func = function(arg1, arg2, ..., argN) {  
    return expression;  
};
```

1. Example

javascript

```
let sum = (a, b) => a + b;  
  
/* Ye is ka chhota form hai:  
  
let sum = function(a, b) {  
    return a + b;  
};  
*/  
  
alert( sum(1, 2) ); // 3
```

`(a, b) => a + b` ka matlab: ek function jo do arguments `a` aur `b` leta hai, `a + b` calculate karta hai aur return karta hai.

2. Alag-alag shapes

Sirf ek argument

Ek hi argument ho to **brackets hata sakte ho**, aur chhota ho jata hai:

```
javascript

let double = n => n * 2;
// lagbhag same: let double = function(n) { return n * 2 }

alert( double(3) ); // 6
```

Koi argument nahi

Argument na ho to **khali brackets zaruri hain**:

```
javascript

let sayHi = () => alert("Hello!");

sayHi();
```

3. Function Expression ki tarah use

Arrow functions ko bilkul Function Expressions ki tarah use kar sakte ho. Jaise **dynamically function banane** ke liye:

```
javascript

let age = prompt("What is your age?", 18);

let welcome = (age < 18) ?
  () => alert('Hello!') :
  () => alert("Greetings!");

welcome();
```

Shuru me arrow functions **anjaan aur kam readable** lag sakte hain, lekin aankhein jaldi aadat daal leti hain. **Simple one-line actions** ke liye bahut convenient hain, jab bahut saare words likhne ka mann na ho.

4. Multiline arrow functions

Ab tak ke arrow functions simple the: $\boxed{=>}$ ke left se arguments liye, right ki expression evaluate karke return kar di.

Kabhi kabhi **complex function** chahiye jisme kai statements ho. Tab unhe **curly braces** `{ }` me likhte hain.

Bada fark: curly braces lagane par `return` **explicitly** likhna padta hai (bilkul normal function ki tarah).

javascript

```
let sum = (a, b) => { // curly brace = multiline function
  let result = a + b;
  return result;      // curly braces ke saath explicit "return" zaruri
};

alert( sum(1, 2) ); // 3
```

More to come

Arrow functions ki aur bhi interesting features hain, jo baad me chapter "**Arrow functions revisited**" me aayengi. Abhi hum inhe **one-line actions** aur **callbacks** ke liye use kar sakte hain.

Summary

Arrow functions simple actions ke liye handy hain, khaaskar one-liners ke liye. Ye **do flavors** me aate hain:

1. Curly braces ke bina: `(...args) => expression`

- Right side ek expression hoti hai. Function use evaluate karke result return karta hai.
- Sirf ek argument ho to brackets hata sakte ho, jaise `n => n*2`.

2. Curly braces ke saath: `(...args) => { body }`

- Kai statements likh sakte ho, lekin kuch return karna ho to **explicit** `return` chahiye.

Practice Task (Answer)

Function Expressions ko arrow functions se replace karo:

javascript

```
function ask(question, yes, no) {
  if (confirm(question)) yes();
  else no();
}

ask(
  "Do you agree?",
  () => alert("You agreed."),
  () => alert("You canceled the execution.")
);
```

Chhota aur saaf lag raha hai na?

Quick Summary

Shape	Syntax	Example
Do ya zyada arguments	<code>(a, b) => expression</code>	<code>(a, b) => a + b</code>
Ek argument	<code>n => expression</code>	<code>n => n * 2</code>
Koi argument nahi	<code>() => expression</code>	<code>() => alert("Hello!")</code>
Multiline	<code>(a, b) => { ...; return x; }</code>	<code>return</code> zaruri

Yaad rakho:

- Curly braces **nahi** = automatic return.
- Curly braces **hain** = `return` khud likhna padega.
- One-liners aur callbacks ke liye best.